COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE010	**SECTION:**	CPE21S2
DATE PERFORMED:	July 17-21, 2026	**DATE SUBMITTED:**	July 21, 2026
NAME:	Jaime Luis M. Demain	**INSTRUCTOR:**	Engr. Jimlord M. Quejado

ACTIVITY NO. 2.1 **Arrays, Pointers and Dynamic Memory Allocation**

1. PROCEDURE OUTPUT

What procedures did you perform?

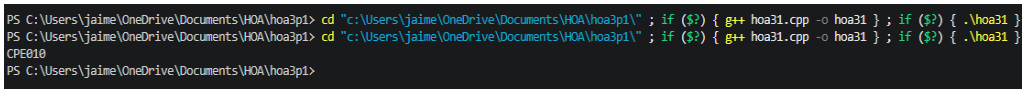
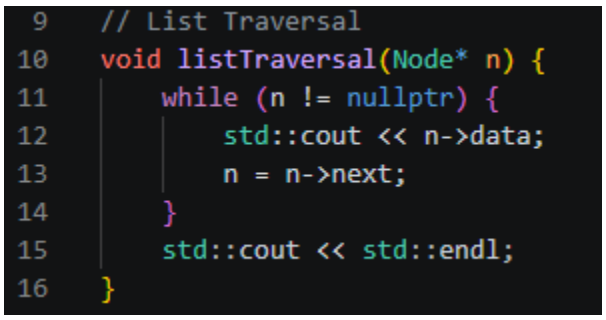
Screenshot	
Observation	The output CPE010 is obtained by manually creating six pointers to Node on the heap memory through the use of the new operator, initializing them with some particular characters, and connecting them together from the head node to the last node with a nullptr. Although this does demonstrate effectively how the nodes connect to their respective neighboring memory locations, making a linked list, this hard-coded implementation is very inefficient, unnecessary, and totally unscalable for bigger sets of data. This method can definitely be improved with the use of helper functions such as a traversal loop and dynamic insertions (insertAtHead, insertAtEnd).

Table 3-1. Output of Initial/Simple Implementation

Traversal	

Insertion at head

```
18  // Insertion at Head
19  void insertAtHead(Node** head_ref, char new_data) {
20      // 1. Allocate memory for the new node
21      Node* new_node = new Node();
22
23      // 2. Put data into new node
24      new_node->data = new_data;
25
26      // 3. Set Next of new node to point to previous Head
27      new_node->next = *head_ref;
28
29      // 4. Reset Head to point to new node
30      *head_ref = new_node;
31  }
```

Insertion at any part of the list

```
33  // Insertion after a specific previous node
34  void insertAfter(Node* prev_node, char new_data) {
35      // 1 & 2. Check if the previous node is null
36      if (prev_node == nullptr) {
37          std::cout << "Previous node cannot be null.\n";
38          return;
39      }
40
41      // 3. Allocate a new node
42      Node* new_node = new Node();
43
44      // 4. Store data in the new node
45      new_node->data = new_data;
46
47      // 5. Point new node to the node previous node was pointing to
48      new_node->next = prev_node->next;
49
50      // 6. Point previous node to the new node
51      prev_node->next = new_node;
52  }
```

Insertion at the end

```
54  // Insertion at the End
55  void insertAtEnd(Node** head_ref, char new_data) {
56      // 1. Allocate new node
57      Node* new_node = new Node();
58
59      // 2. Dereference to the head node (used to traverse)
60      Node* last = *head_ref;
61
62      // 3. Store data in new node
63      new_node->data = new_data;
64
65      // 4. Point next of new node to NULL
66      new_node->next = nullptr;
67
68      // If the Linked List is empty, make new_node the head
69      if (*head_ref == nullptr) {
70          *head_ref = new_node;
71          return;
72      }
73
74      // 5. Traverse the list until next of the node is null
75      while (last->next != nullptr) {
76          last = last->next;
77      }
78
79      // 6. Point the next of the current node to the new node
80      last->next = new_node;
81  }
```

Deletion of a node	

```
// Deletion of a node containing target data
void deleteNode(Node** head_ref, char key) {
    Node* temp = *head_ref;
    Node* prev = nullptr;

    // If head node itself holds the key to be deleted
    if (temp != nullptr && temp->data == key) {
        *head_ref = temp->next; // Change head
        delete temp;            // Free memory
        return;
    }

    // 1. Find previous node of the node to be deleted
    while (temp != nullptr && temp->data != key) {
        prev = temp;
        temp = temp->next;
    }

    // If key was not present in linked list
    if (temp == nullptr) return;

    // 2. Change the next of previous node
    prev->next = temp->next;

    // 3. Free memory for the node to be deleted
    delete temp;
}
```

Table 3-2. Code for the List Operations

a.	Source Code	

```
// Build initial list "CPE101" using insertAtEnd
insertAtEnd(&head, 'C');
insertAtEnd(&head, 'P');
insertAtEnd(&head, 'E');
insertAtEnd(&head, '1');
insertAtEnd(&head, '0');
insertAtEnd(&head, '1');
```

	Console	

```
(a) Initial List: CPE101
```

b.	Source Code	`// (b) Insert element 'G' at head (output -> GCPE101)`
		`insertAtHead(&head, 'G');`
		`std::cout << "(b) After inserting 'G' at start: ";`
		`listTraversal(head); // Output: GCPE101`

	Console	`(b) After inserting 'G' at start: GCPE101`

c.	Source Code	`// (c) Insert element 'E' after previous node 'P' (output -> GCPEE101)`
		`Node* temp = head;`
		`while (temp != nullptr && temp->data != 'P') {`
		`    temp = temp->next;`
		`}`
		`insertAfter(temp, 'E');`
		`std::cout << "(c) After inserting 'E' after 'P': ";`
		`listTraversal(head); // Output: GCPEE101`

	Console	`(c) After inserting 'E' after 'P': GCPEE101`

d.	Source Code	`// (d) Delete node containing 'C'`
		`deleteNode(&head, 'C');`
		`std::cout << "(d) After deleting 'C': ";`
		`listTraversal(head); // Output: GPEE101`

	Console	`(d) After deleting 'C': GPEE101`

e.	Source Code	`// (e) Delete node containing 'P'`
		`deleteNode(&head, 'P');`
		`std::cout << "(e) After deleting 'P': ";`
		`listTraversal(head); // Output: GEE101`

	Console	`(e) After deleting 'P': GEE101`

f.	Source Code	`// (f) Show final elements in the list (output -> GEE101)`
		`std::cout << "(f) Final List Elements: ";`
		`listTraversal(head); // Output: GEE101`

	Console	`(f) Final List Elements: GEE101`

Table 2-3

Modifications	`for(int i = 0; i < j; i++){ //loop A`
	`Student *ptr = new Student(namesList[i], ageList[i]);`
	`studentList[i] = *ptr;`
	`//modification;`
	`delete ptr;`
	`}`

Observation	The memory leak detected at Loop A was fixed by inserting delete ptr. Since the original dynamically allocated object Student (ptr) has served its purpose of transferring the initialized properties to the static studentList array, not releasing it led to orphan memory on the heap. Using delete operator guarantees that the destructor will be invoked as soon as the value has been transferred for every dynamically allocated new. This guarantees that the heap memory is managed efficiently and leads to a balanced number of constructors and destructors, 10 each at runtime exit.

Table 3-3. Code and Analysis for Singly Linked Lists

```cpp
#include <iostream>

class Node {
public:
    char data;
    Node* next;
    Node* prev; // Added pointer to previous node
};

// List Traversal (Forward)
void listTraversal(Node* n) {
    while (n != nullptr) {
        std::cout << n->data;
        n = n->next;
    }
    std::cout << std::endl;
}

// Insertion at Head
void insertAtHead(Node** head_ref, char new_data) {
    // 1. Allocate memory for new node
    Node* new_node = new Node();

    // 2. Assign data and set pointers
    new_node->data = new_data;
    new_node->next = *head_ref;
    new_node->prev = nullptr; // New head's prev is always null

    // 3. Update old head's prev pointer if list was not empty
    if (*head_ref != nullptr) {
        (*head_ref)->prev = new_node;
    }

    // 4. Reset head to point to new node
    *head_ref = new_node;
}

// Insertion after a specific previous node
void insertAfter(Node* prev_node, char new_data) {
    if (prev_node == nullptr) {
        std::cout << "Previous node cannot be null.\n";
        return;
    }

    // 1. Allocate new node and store data
    Node* new_node = new Node();
    new_node->data = new_data;

```

In this screenshot, the definition of the structure for doubly linked list and the basic operations on it is provided. The Node class is defined with data fields along with two pointers next and prev to allow for traversal in both directions. listTraversal traverses nodes one-by-one and prints their data from head to tail. insertAtHead takes care of insertion at the head with allocation of memory and proper setting of pointers for the newly inserted node and the previously stored head node. At last, insertAfter starts the process of insertion of a node with proper validation of the previous node.

```
49      // 2. Set pointers for new node
50      new_node->next = prev_node->next;
51      new_node->prev = prev_node;
52
53      // 3. Set new node as prev of prev_node's next node
54      if (prev_node->next != nullptr) {
55          prev_node->next->prev = new_node;
56      }
57
58      // 4. Point prev_node's next to new node
59      prev_node->next = new_node;
60  }
61
62  // Insertion at the End
63  void insertAtEnd(Node** head_ref, char new_data) {
64      Node* new_node = new Node();
65      Node* last = *head_ref;
66
67      new_node->data = new_data;
68      new_node->next = nullptr;
69
70      // If list is empty, new node becomes head
71      if (*head_ref == nullptr) {
72          new_node->prev = nullptr;
73          *head_ref = new_node;
74          return;
75      }
76
77      // Traverse to the last node
78      while (last->next != nullptr) {
79          last = last->next;
80      }
81
82      // Link last node and new node
83      last->next = new_node;
84      new_node->prev = last;
85  }
86
87  // Deletion of a node containing target data
88  void deleteNode(Node** head_ref, char key) {
89      Node* temp = *head_ref;
90
91      // Search for the node to delete
92      while (temp != nullptr && temp->data != key) {
93          temp = temp->next;
94      }
95
```

This screenshot completes the insertion of middle and tail before implementing the node deletion process. In the function insertAfter, the process is completed to connect a new middle node by changing the pointers in both adjacent nodes. In the function insertAtEnd, the process of adding nodes to the tail of the list is accomplished while considering various cases such as if the list is empty or not and normal loops for finding the last node and connecting it. Finally, in the function deleteNode, the process is started by creating a temp pointer.

```
    // Key not found
    if (temp == nullptr) return;

    // If node to be deleted is head node
    if (*head_ref == temp) {
        *head_ref = temp->next;
    }

    // Change next only if node to be deleted is NOT the last node
    if (temp->next != nullptr) {
        temp->next->prev = temp->prev;
    }

    // Change prev only if node to be deleted is NOT the first node
    if (temp->prev != nullptr) {
        temp->prev->next = temp->next;
    }

    // Free memory
    delete temp;
}

int main() {
    Node* head = nullptr;

    // Build initial list "CPE101"
    insertAtEnd(&head, 'C');
    insertAtEnd(&head, 'P');
    insertAtEnd(&head, 'E');
    insertAtEnd(&head, '1');
    insertAtEnd(&head, '0');
    insertAtEnd(&head, '1');

    // (a) Initial Traversal
    std::cout << "(a) Initial List: ";
    listTraversal(head); // Output: CPE101

    // (b) Insert element 'G' at head (output -> GCPE101)
    insertAtHead(&head, 'G');
    std::cout << "(b) After inserting 'G' at start: ";
    listTraversal(head); // Output: GCPE101

    // (c) Insert element 'E' after previous node 'P' (output -> GCPEE101)
    Node* temp = head;
    while (temp != nullptr && temp->data != 'P') {
        temp = temp->next;
    }
```

This screenshot covers the actual deletion of nodes and sets up the execution of the program. The deletion of nodes is handled safely by modifying the boundary pointer of neighboring nodes – either by updating the *head_ref if deleting the first node, or by linking temp->next->prev and temp->prev->next within the interior nodes – before using delete temp to release memory from the heap while keeping the list chain connected. Within the main function, an uninitialized head pointer is used to insert "CPE101" through insertAtEnd. The driver now starts to execute Task (a) traversal, Task (b) insertion of 'G' to create "GCPE101", and searches for node 'P' for Task (c).

```
    insertAfter(temp, 'E');
    std::cout << "(c) After inserting 'E' after 'P': ";
    listTraversal(head); // Output: GCPEE101

    // (d) Delete node containing 'C'
    deleteNode(&head, 'C');
    std::cout << "(d) After deleting 'C': ";
    listTraversal(head); // Output: GPEE101

    // (e) Delete node containing 'P'
    deleteNode(&head, 'P');
    std::cout << "(e) After deleting 'P': ";
    listTraversal(head); // Output: GEE101

    // (f) Show final elements in the list (output -> GEE101)
    std::cout << "(f) Final List Elements: ";
    listTraversal(head); // Output: GEE101

    // Memory Cleanup
    while (head != nullptr) {
        Node* next_node = head->next;
        delete head;
        head = next_node;
    }

    return 0;
}
```

Conclusion: On this activity, I learned about dynamic memory allocation and manipulation of pointers through conversion of hard-coded singly linked lists to modular singly, doubly and circular linked lists in C++. The initial exercise in Table 3-1 showed me the limitations of allocating memory individually for each node on the heap and the need for scalable processes. From Table 3-2 and Table 3-3, I was able to manipulate nodes systematically using modular lists by performing traversal, insertions at head and end, targeted insertions and deletions to get the

target output ending in "GEE101". Re-designing the program for Table 3-4 as a doubly linked list made it clear to me how the bidirectionality of pointers (next and prev) makes insertion in the middle and deletion of nodes simple. Additionally, the supplementary activity made me use these skills by designing a real-time application of a doubly linked circular list to create an interactive music playlist program. In summary, I achieved all target terminal outputs and made sure that I released dynamically allocated memory properly; in the future, I shall work on improving my efficiency in handling pointers' edge cases in node deletions.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

```
#include <iostream>
#include <string>
#include "playlist.h"

void printMenu() {
    std::cout << "\n========== PLAYLIST MENU ==========\n";
    std::cout << "[1] Display Playlist\n";
    std::cout << "[2] Play All Songs\n";
    std::cout << "[3] Next Song\n";
    std::cout << "[4] Previous Song\n";
    std::cout << "[5] Insert Song\n";
    std::cout << "[6] Remove Song\n";
    std::cout << "[0] Exit\n";
    std::cout << "Enter Choice: ";
}

int main() {
    SongNode<std::string>* playlist = nullptr;
    SongNode<std::string>* currentTrack = nullptr;

    // Optional: Pre-populate with a few initial songs
    addSong(std::string("Through the Years"), std::string("Kenny
Rogers"), &playlist);
    addSong(std::string("Am I Wrong"), std::string("Nico & Vinz"),
&playlist);
    addSong(std::string("Way Back Home"), std::string("SHAUN"),
&playlist);

    currentTrack = playlist; // Set pointer to start of playlist
```

```
int choice = -1;

while (choice != 0) {
    printMenu();
    std::cin >> choice;
    std::cin.ignore(); // Clear newline character left by std::cin

    std::cout << "\n";

    switch (choice) {
        case 1:
            displayPlaylist(playlist);
            break;

        case 2:
            playAllSongs(playlist);
            break;

        case 3:
            if (currentTrack != nullptr) {
                nextTrack(&currentTrack);
            } else {
                std::cout << "Playlist is empty.\n";
            }
            break;

        case 4:
            if (currentTrack != nullptr) {
                prevTrack(&currentTrack);
            } else {
                std::cout << "Playlist is empty.\n";
            }
            break;

        case 5: {
            std::string title, artist;
            std::cout << "Enter Song Title: ";
```

```
                std::getline(std::cin, title);
                std::cout << "Enter Artist Name: ";
                std::getline(std::cin, artist);

                addSong(title, artist, &playlist);

                // If this is the first song added, initialize
currentTrack pointer
                if (currentTrack == nullptr) {
                    currentTrack = playlist;
                }

                std::cout << "Added \"" << title << "\" to the
playlist.\n";
                break;
            }

            case 6: {
                if (playlist == nullptr) {
                    std::cout << "Playlist is empty. Nothing to
remove.\n";
                    break;
                }

                std::string title;
                std::cout << "Enter Song Title to Remove: ";
                std::getline(std::cin, title);

                // Adjust currentTrack pointer if deleting the active
track
                if (currentTrack != nullptr && currentTrack->title ==
title) {
                    currentTrack = currentTrack->next;
                }

                deleteSong(title, &playlist);

                // If playlist becomes completely empty after deletion
```

```
                if (playlist == nullptr) {
                    currentTrack = nullptr;
                }
                break;
            }

            case 0:
                std::cout << "Exiting playlist application. Cleaning
memory...\n";
                deletePlaylist(&playlist);
                currentTrack = nullptr;
                break;

            default:
                std::cout << "Invalid choice. Please try again.\n";
                break;
        }
    }

    return 0;
}
```

Code for main.cpp file

```
#ifndef PLAYLIST_H
#define PLAYLIST_H

// Node template for a song item in the playlist
template <typename T>
class SongNode {
public:
    T title;
    T artist;
    SongNode<T>* next = nullptr;
    SongNode<T>* prev = nullptr;
};

// Helper function to create a new song node dynamically
```

```cpp
template <typename T>
SongNode<T>* createSongNode(T songTitle, T songArtist) {
    SongNode<T>* newNode = new SongNode<T>();
    newNode->title = songTitle;
    newNode->artist = songArtist;
    newNode->next = nullptr;
    newNode->prev = nullptr;
    return newNode;
}

// Function to display playlist items
template <typename T>
void displayPlaylist(SongNode<T>* head) {
    if (head == nullptr) {
        std::cout << "Playlist is empty.\n";
        return;
    }

    SongNode<T>* current = head;
    std::cout << "Playlist:\n";
    do {
        std::cout << current->title << "\n";
        current = current->next;
    } while (current != head);
}

// Traverse and play all songs ONCE in a loop (as requested)
template <typename T>
void playAllSongs(SongNode<T>* head) {
    if (head == nullptr) {
        std::cout << "The playlist is empty.\n";
        return;
    }

    SongNode<T>* current = head;
    std::cout << "Playing All Songs (Circular Loop)\n";
    do {
        std::cout << "Playing: \"" << current->title << "\" by " <<
```

```
current->artist << "\n";
        current = current->next;
    } while (current != head); // Stops after completing one full circle
    std::cout << "\n=========================================\n";
}

// Add a song to the end (maintaining circular links)
template <typename T>
void addSong(T title, T artist, SongNode<T>** head) {
    SongNode<T>* newNode = createSongNode(title, artist);

    // If playlist is empty
    if (*head == nullptr) {
        *head = newNode;
        newNode->next = *head;
        newNode->prev = *head;
        return;
    }

    // Get the tail node (which is head's prev in a circular list)
    SongNode<T>* tail = (*head)->prev;

    // Connect new node between tail and head
    tail->next = newNode;
    newNode->prev = tail;
    newNode->next = *head;
    (*head)->prev = newNode;
}

// Move to next track
template <typename T>
void nextTrack(SongNode<T>** current) {
    if (*current != nullptr) {
        *current = (*current)->next;
        std::cout << ">> Next Track: \"" << (*current)->title << "\" by "
<< (*current)->artist << "\n";
    }
}
```

```
// Move to previous track
template <typename T>
void prevTrack(SongNode<T>** current) {
    if (*current != nullptr) {
        *current = (*current)->prev;
        std::cout << "<< Previous Track: \"" << (*current)->title << "\"
by " << (*current)->artist << "\n";
    }
}

// Delete a song by title in circular list
template <typename T>
void deleteSong(T title, SongNode<T>** head) {
    if (*head == nullptr) return;

    SongNode<T>* current = *head;
    bool found = false;

    // Search circular list
    do {
        if (current->title == title) {
            found = true;
            break;
        }
        current = current->next;
    } while (current != *head);

    if (!found) {
        std::cout << "Song \"" << title << "\" not found in playlist.\n";
        return;
    }

    // Case 1: Only 1 song in list
    if (current->next == current) {
        delete current;
        *head = nullptr;
        std::cout << "Successfully deleted \"" << title << "\". Playlist
```

```
is now empty.\n";
        return;
    }

    // Case 2: Deleting head node
    if (current == *head) {
        *head = current->next;
    }

    // Fix surrounding circular pointers
    current->prev->next = current->next;
    current->next->prev = current->prev;

    delete current;
    std::cout << "Successfully deleted \"" << title << "\" from the
playlist.\n";
}

// Deallocate circular playlist
template <typename T>
void deletePlaylist(SongNode<T>** head) {
    if (*head == nullptr) return;

    SongNode<T>* current = *head;
    // Break circular connection first to prevent infinite loop during
deletion
    (*head)->prev->next = nullptr;

    while (current != nullptr) {
        SongNode<T>* temp = current;
        current = current->next;
        delete temp;
    }
    *head = nullptr;
    std::cout << "All songs cleared from playlist.\n";
}
#endif
```

Code for header file

```
PS C:\Users\jaime\OneDrive\Documents\HOA\hoa3p1> cd "c:\Users\jaime\OneDrive\Documents\HOA\hoa3p1\" ; if ($?) { g++ hoa32supp.cpp -o hoa32supp } ; if ($?) { .\hoa32supp }

========== PLAYLIST MENU ==========
[1] Display Playlist
[2] Play All Songs
[3] Next Song
[4] Previous Song
[5] Insert Song
[6] Remove Song
[0] Exit
Enter Choice: 1

Playlist:
Through the Years
Am I Wrong
Way Back Home

========== PLAYLIST MENU ==========
[1] Display Playlist
[2] Play All Songs
[3] Next Song
[4] Previous Song
[5] Insert Song
[6] Remove Song
[0] Exit
Enter Choice: 2

Playing All Songs (Circular Loop)
Playing: "Through the Years" by Kenny Rogers
Playing: "Am I Wrong" by Nico & Vinz
Playing: "Way Back Home" by SHAUN

======================================

========== PLAYLIST MENU ==========
[1] Display Playlist
[2] Play All Songs
[3] Next Song
[4] Previous Song
[5] Insert Song
[6] Remove Song
[0] Exit
Enter Choice: 3
```

```
>> Next Track: "Am I Wrong" by Nico & Vinz

========== PLAYLIST MENU ==========
[1] Display Playlist
[2] Play All Songs
[3] Next Song
[4] Previous Song
[5] Insert Song
[6] Remove Song
[0] Exit
Enter Choice: 5

Enter Song Title: Yeah 3x
Enter Artist Name: Chris Brown
Added "Yeah 3x" to the playlist.

========== PLAYLIST MENU ==========
[1] Display Playlist
[2] Play All Songs
[3] Next Song
[4] Previous Song
[5] Insert Song
[6] Remove Song
[0] Exit
Enter Choice: 1

Playlist:
Through the Years
Am I Wrong
Way Back Home
Yeah 3x

========== PLAYLIST MENU ==========
[1] Display Playlist
[2] Play All Songs
[3] Next Song
[4] Previous Song
[5] Insert Song
[6] Remove Song
[0] Exit
Enter Choice: 2
```

```
Playing All Songs (Circular Loop)
Playing: "Through the Years" by Kenny Rogers
Playing: "Am I Wrong" by Nico & Vinz
Playing: "Way Back Home" by SHAUN
Playing: "Yeah 3x" by Chris Brown

====================================

========== PLAYLIST MENU ==========
[1] Display Playlist
[2] Play All Songs
[3] Next Song
[4] Previous Song
[5] Insert Song
[6] Remove Song
[0] Exit
Enter Choice: cd "c:\Users\jaime\OneDrive\Documents\HOA\hoa3p1\" ; if ($?) { g++ tempCodeRunnerFile.cpp -o tempCodeRunnerFile } ; if ($?) { .\tempCodeRunnerFile }

Exiting playlist application. Cleaning memory...
All songs cleared from playlist.
```

Output of the program

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **Gemini**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Code assistance for in Doubly Linked List part in procedure, and supplementary activitiy. It helped me to gain insights on how I can change the single linked list version of code into double linked list one, and finalize my whole output in supplementary activity. Then, improvement of grammar in observation and conclusion.**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **I used gemini to visualize how I can convert a singly linked list version of code into doubly linked one. Then, I used AI to help me understand the supplementary activity and helped me accomplish it, as it is kind of complicated for me to finish the whole output. Plus, I used the AI to improve my grammar in forming an observations and conclusion and correct any mistakes.**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.